CS 301
High-Performance Computing

Assignment 02 - Matrix Multiplication

Nilesh Mori (202301473)
Harsh Maheriya (202301470)

February 17, 2026

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 Hardware Details for LAB207 PCs . . . 3
2.2 Hardware Details for HPC Cluster (Node gics1) . . . 4

3 Problem Description **5**

4 Benchmarking Methodology **5**

5 Graphical Results **6**
5.1 Algorithm: ijk . . . 6
5.1.1 Using Lab PC . . . 6
5.1.2 Using HPC Cluster . . . 7
5.2 Algorithm: ikj . . . 7
5.2.1 Using Lab PC . . . 7
5.2.2 Using HPC Cluster . . . 8
5.3 Algorithm: jik . . . 8
5.3.1 Using Lab PC . . . 8
5.3.2 Using HPC Cluster . . . 9
5.4 Algorithm: jki . . . 9
5.4.1 Using Lab PC . . . 9
5.4.2 Using HPC Cluster . . . 10
5.5 Algorithm: kij . . . 10
5.5.1 Using Lab PC . . . 10
5.5.2 Using HPC Cluster . . . 11
5.6 Algorithm: kji . . . 11
5.6.1 Using Lab PC . . . 11
5.6.2 Using HPC Cluster . . . 12
5.7 Algorithm: Transpose . . . 12
5.7.1 Using Lab PC . . . 12
5.7.2 Using HPC Cluster . . . 13
5.8 Algorithm: Block . . . 13
5.8.1 Using Lab PC . . . 13
5.8.2 Using HPC Cluster . . . 14

6 Conclusion **15**

1 Introduction

In this assignment, we explore the performance characteristics of various matrix multiplication algorithms on different hardware platforms: a local Lab PC and an HPC Cluster. The objective is to understand how different loop orderings (ijk permutations), matrix transposition, and blocking strategies impact execution time and computational efficiency (MFLOPS) as a function of problem size. By analyzing the performance, we aim to identify memory access patterns, cache utilization effects, and the overall efficiency of each algorithm.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x
- L1d cache: 288K
- L1i cache: 192K

- L2 cache: 7.5M
- L3 cache: 18M
- NUMA node0 CPU(s): 0-11

2.2 Hardware Details for HPC Cluster (Node gics1)

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 16
- On-line CPU(s) list: 0-15
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 1288.726
- BogoMIPS: 5204.73
- Virtualization: VT-x
- L1d cache: 32K
- L1i cache: 32K
- L2 cache: 256K
- L3 cache: 20480K
- NUMA node0 CPU(s): 0-7
- NUMA node1 CPU(s): 8-15

3 Problem Description

This assignment focuses on the performance analysis of matrix multiplication, a fundamental operation in high-performance computing. The core problem is to multiply two square matrices, $A \times B = C$ where A, B, and C are $N \times N$ matrices of double-precision floating-point numbers. We investigate three main approaches:

1. **Conventional Matrix Multiplication (6 i-j-k variants):** We implement all six permutations of the triply-nested loops (ijk, ikj, jik, jki, kij, kji) to observe how loop ordering affects performance, primarily due to cache access patterns.
2. **Matrix Multiplication using Transpose:** This variant involves transposing one of the input matrices to improve cache locality and potentially reduce cache misses.
3. **Block Matrix Multiplication:** This strategy divides the matrices into smaller blocks, performing multiplication on these blocks to further enhance data locality and utilize caches more effectively.

The goal is to measure and analyze the execution time and MFLOPS (Millions of Floating-point Operations per Second) for each algorithm as a function of the problem size, N. Problem sizes range from 2^2 to 2^{12} (4 to 4096).

4 Benchmarking Methodology

For each algorithm and on both hardware platforms (Lab PC and HPC Cluster), the following methodology was applied:

- **Serial Code Development:** All algorithms were implemented in C++ as serial code.
- **Data Collection:** For each problem size (N), the code was executed 5 times (or 1 time for very slow algorithms) to account for fluctuations in runtime.
- **Problem Sizes:** Experiments were conducted for matrix sizes N from 2^2 to 2^{12}.
- **Performance Metrics:**
  - **Execution Time:** Measured in seconds.
  - **MFLOPS/sec:** Calculated based on $2N^3$ floating-point operations.
- **Hardware:** All runs were performed on two machines: a local Lab PC (LAB207) and an HPC Cluster (Node gics1).

5 Graphical Results

5.1 Algorithm: ijk

5.1.1 Using Lab PC

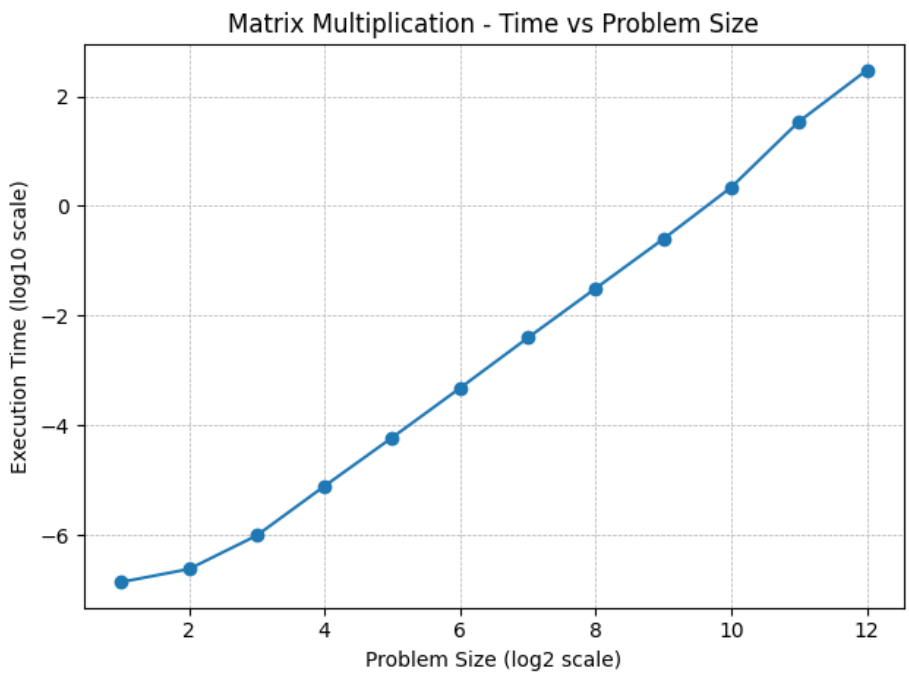

Figure 1: Execution Time vs. Problem Size for ijk algorithm (Lab PC).

5.1.2 Using HPC Cluster

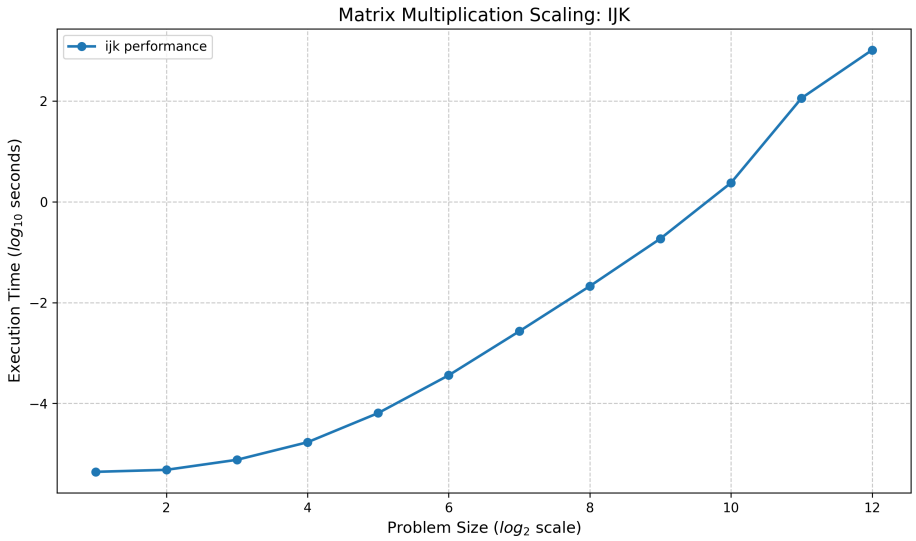

Figure 2: Execution Time vs. Problem Size for ijk algorithm (HPC Cluster).

5.2 Algorithm: ikj

5.2.1 Using Lab PC

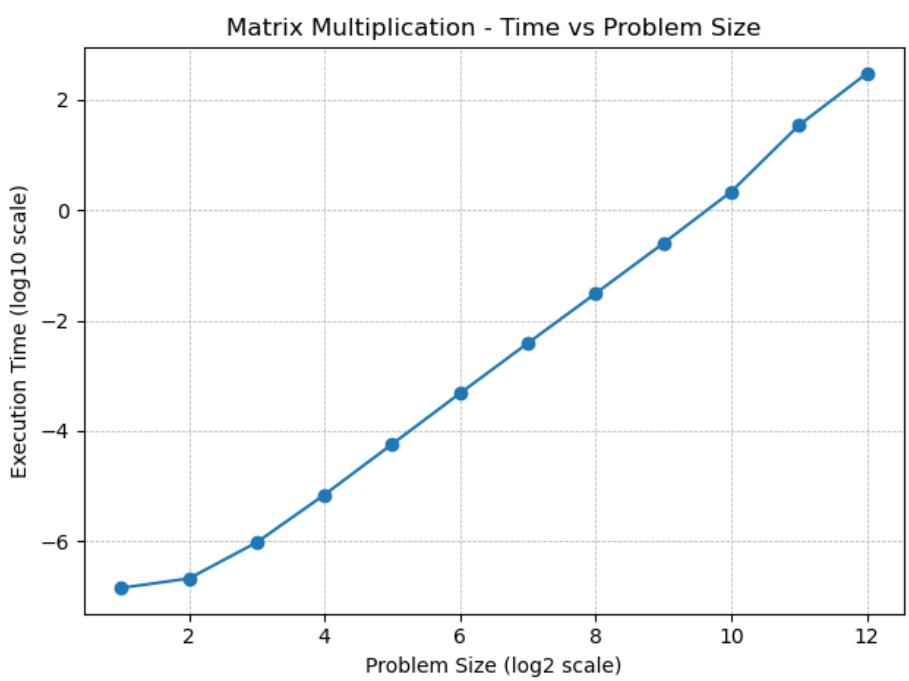

Figure 3: Execution Time vs. Problem Size for ikj algorithm (Lab PC).

5.2.2 Using HPC Cluster

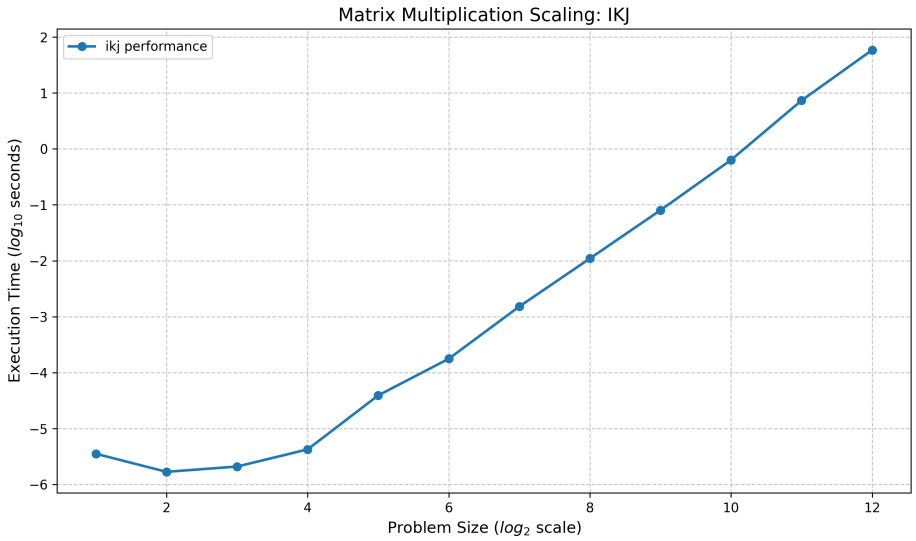

Figure 4: Execution Time vs. Problem Size for ikj algorithm (HPC Cluster).

5.3 Algorithm: jik

5.3.1 Using Lab PC

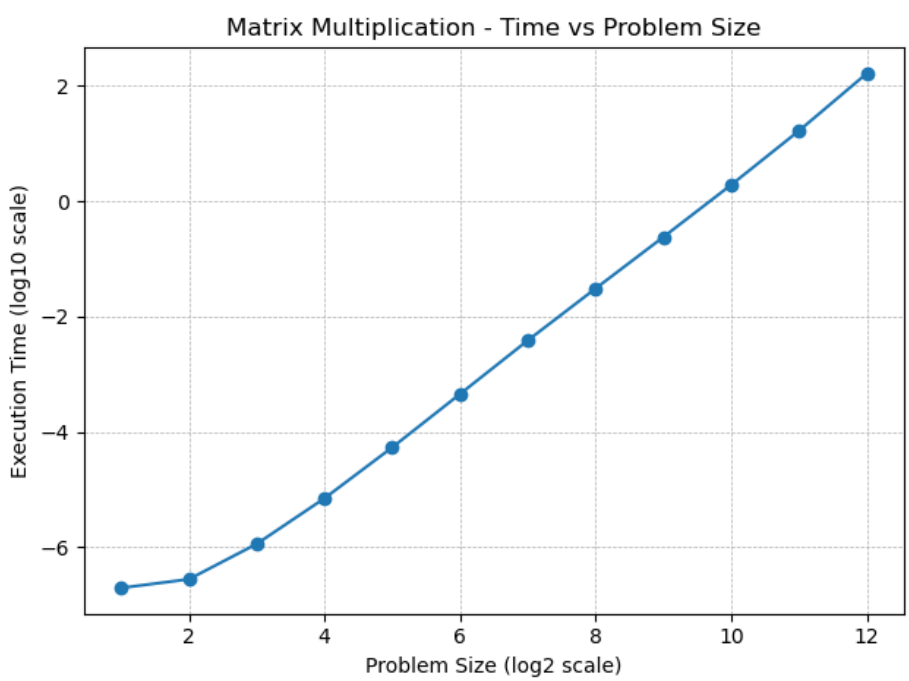

Figure 5: Execution Time vs. Problem Size for jik algorithm (Lab PC).

5.3.2 Using HPC Cluster

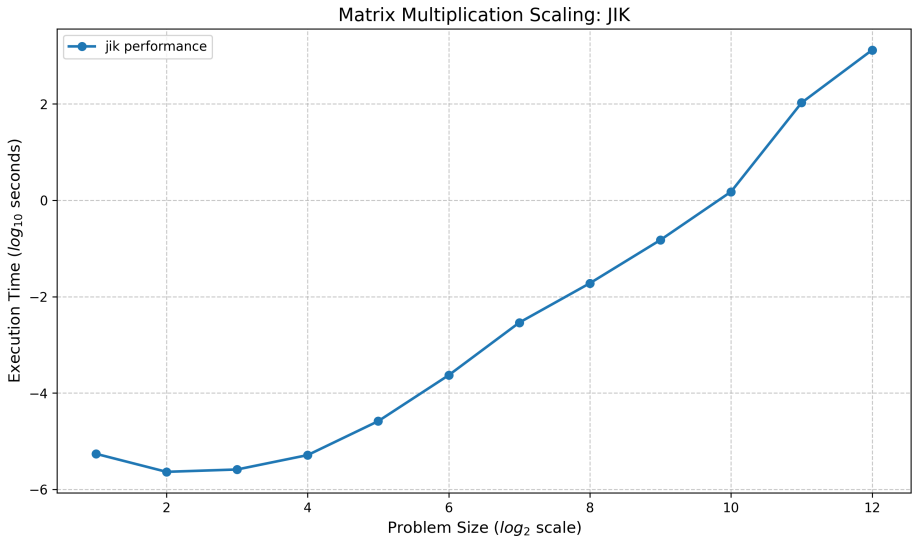

Figure 6: Execution Time vs. Problem Size for jik algorithm (HPC Cluster).

5.4 Algorithm: jki

5.4.1 Using Lab PC

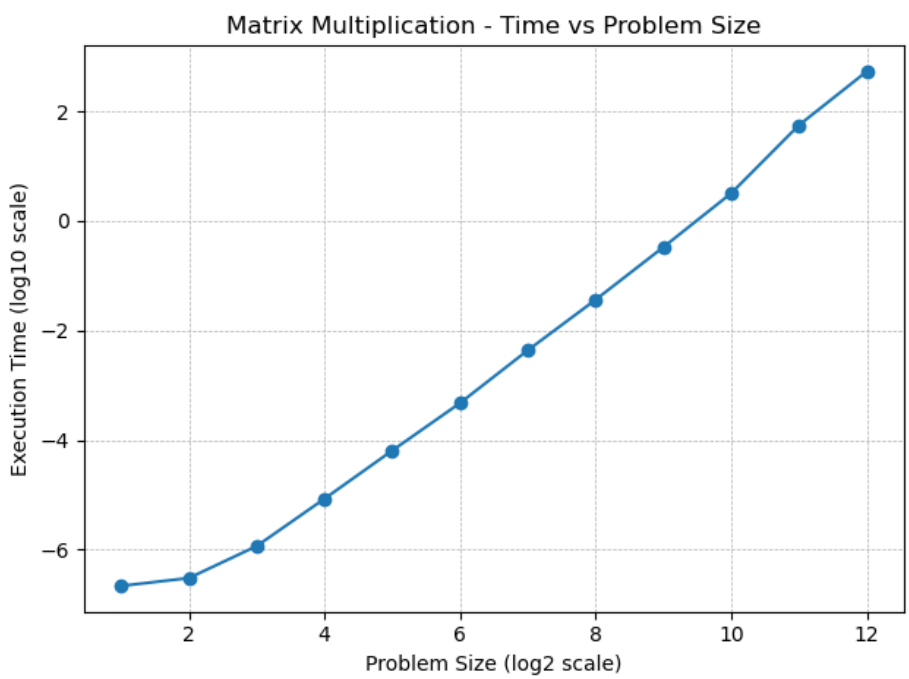

Figure 7: Execution Time vs. Problem Size for jki algorithm (Lab PC).

5.4.2 Using HPC Cluster

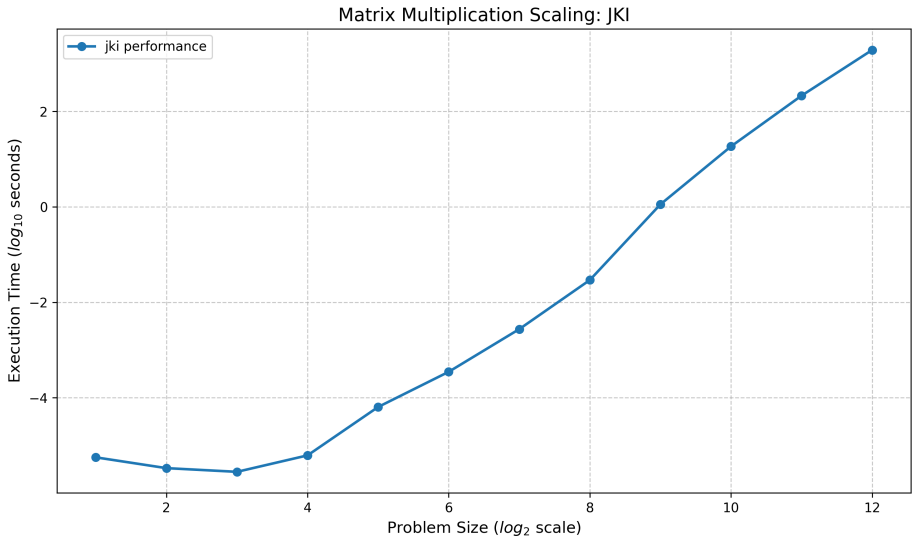

Figure 8: Execution Time vs. Problem Size for jki algorithm (HPC Cluster).

5.5 Algorithm: kij

5.5.1 Using Lab PC

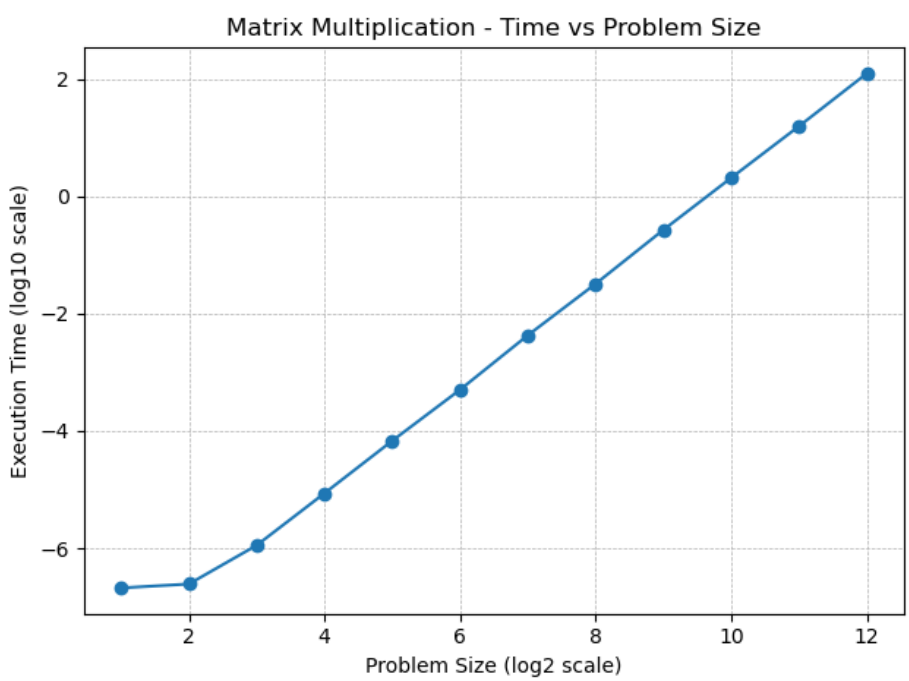

Figure 9: Execution Time vs. Problem Size for kij algorithm (Lab PC).

5.5.2 Using HPC Cluster

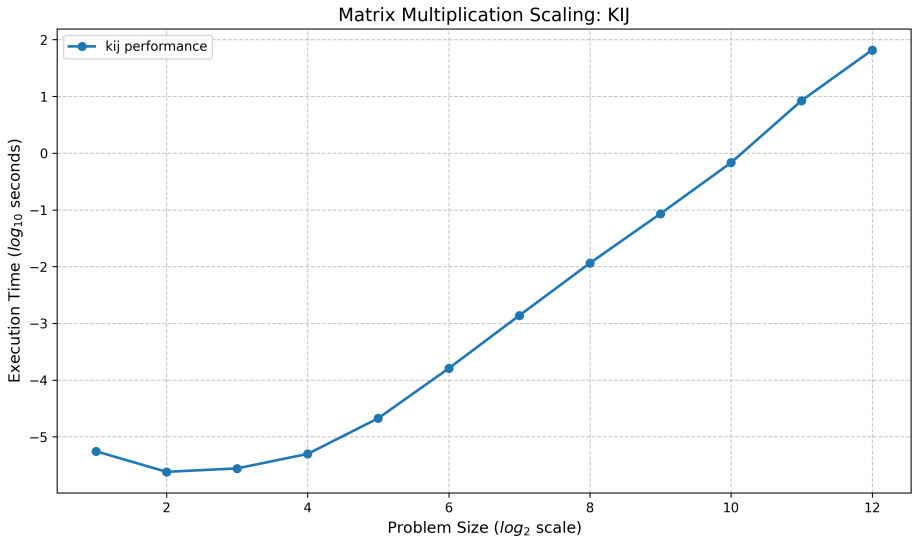

Figure 10: Execution Time vs. Problem Size for kij algorithm (HPC Cluster).

5.6 Algorithm: kji

5.6.1 Using Lab PC

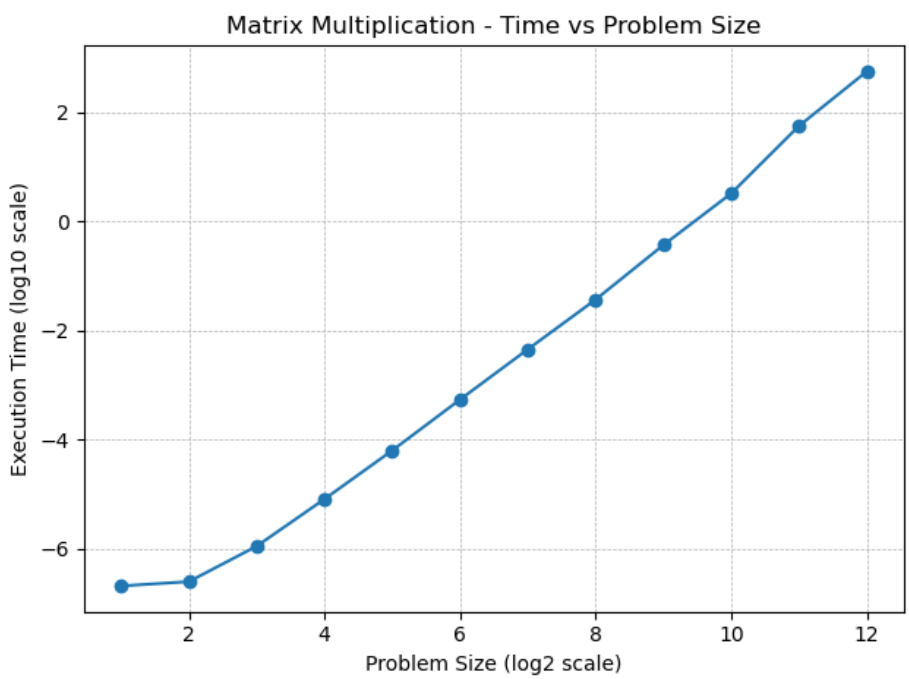

Figure 11: Execution Time vs. Problem Size for kji algorithm (Lab PC).

5.6.2 Using HPC Cluster

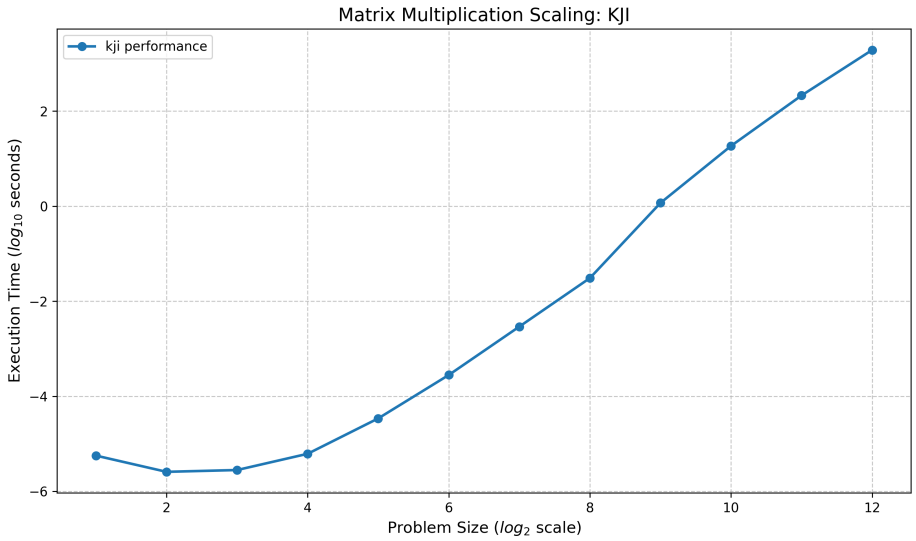

Figure 12: Execution Time vs. Problem Size for kji algorithm (HPC Cluster).

5.7 Algorithm: Transpose

5.7.1 Using Lab PC

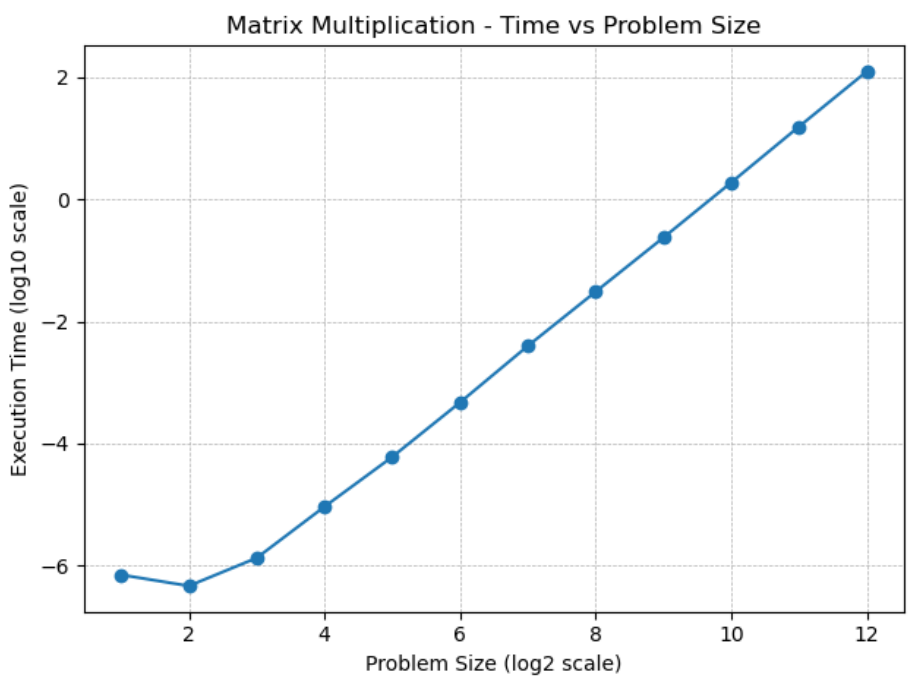

Figure 13: Execution Time vs. Problem Size for Transpose algorithm (Lab PC).

5.7.2 Using HPC Cluster

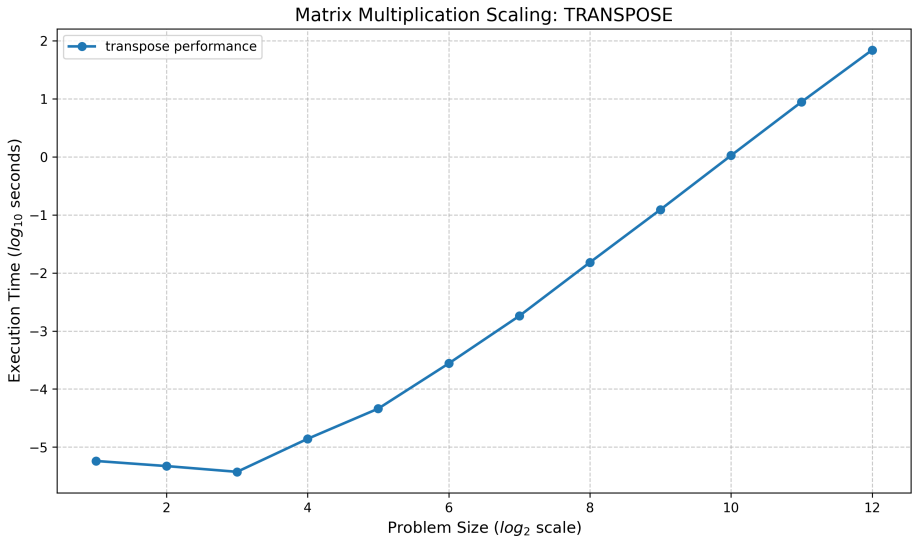

Figure 14: Execution Time vs. Problem Size for Transpose algorithm (HPC Cluster).

5.8 Algorithm: Block

5.8.1 Using Lab PC

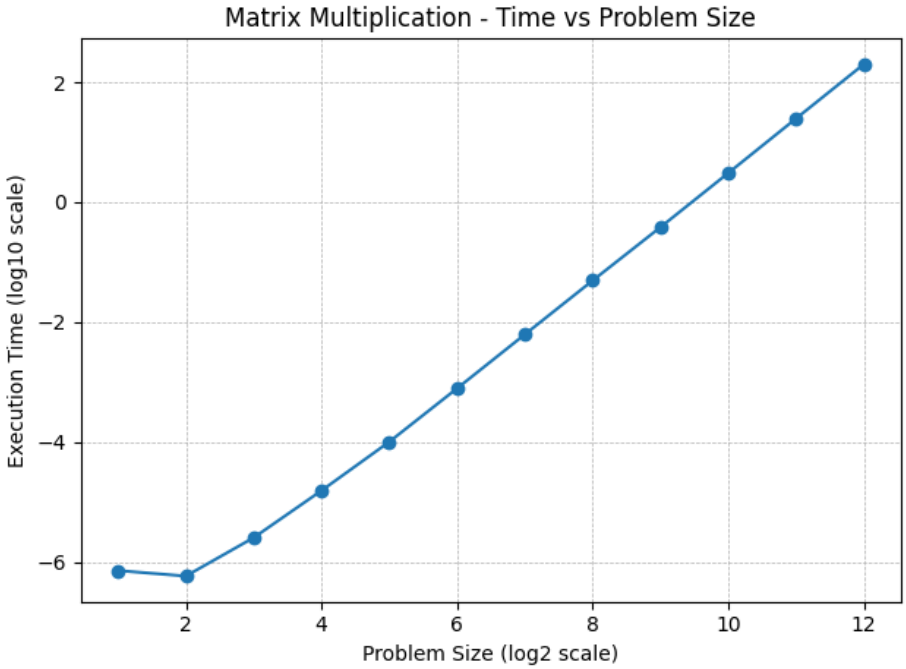

Figure 15: Execution Time vs. Problem Size for Block algorithm (Lab PC).

5.8.2 Using HPC Cluster

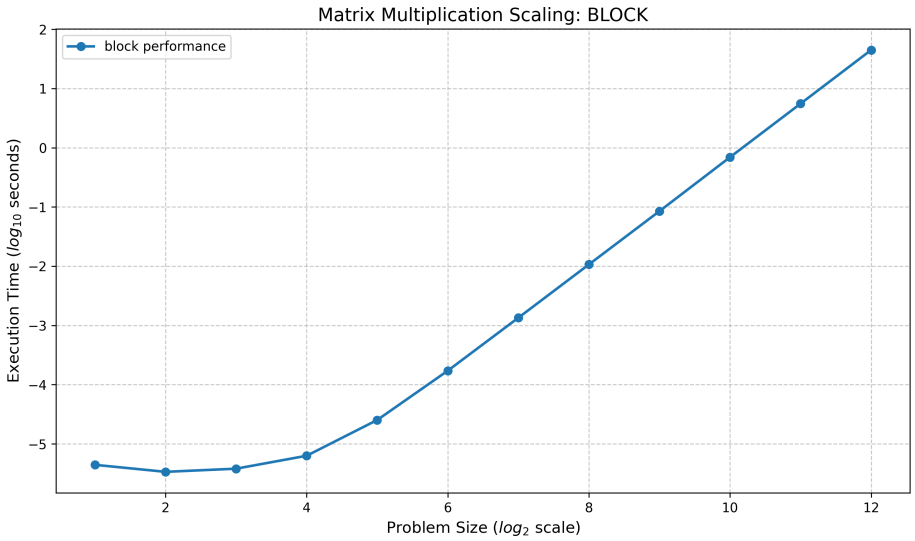

Figure 16: Execution Time vs. Problem Size for Block algorithm (HPC Cluster).

6 Conclusion

This report presented a performance analysis of various matrix multiplication algorithms. We observed significant differences in execution time and MFLOPS across different loop orderings, as well as with the transpose and block optimization strategies. Generally, algorithms that exhibit better cache locality (e.g., ‘ikj‘, ‘kij‘, ‘block‘, ‘transpose‘) tend to perform significantly better than those with poor cache utilization (e.g., ‘jik‘, ‘jki‘, ‘kji‘), especially as the problem size increases. The superior performance of block matrix multiplication and the transpose method highlights the critical importance of memory access patterns and effective cache usage in achieving high performance.